



FOR JAVA DEVELOPERS MOVING TO NESTJS

Four log lines. Two ids.

Spring Boot to NestJS, episode 36: logging

Structured logs tell you what happened. Request context tells you which execution it belonged to.

Both stacks emit JSON with one switch. Neither switch invents a request id.

Spring's MDC is thread local by default: a `CompletableFuture` and an `@Async` method lost the id until a `TaskDecorator` copied it, and the common pool still had nothing.

Nest's `ConsoleLogger` carries no request context. `nestjs-pino`, through `AsyncLocalStorage`, carried it on all four lines.

In scope

- SLF4J and Logback to Nest's Logger
- JSON logs
- MDC and `TaskDecorator`
- `AsyncLocalStorage` via `nestjs-pino`

Not in scope

- Traces (episode 37)
- Log shipping and retention

At the time of recording Nest's `ConsoleLogger` has a `json` option.

01 The measurements

SPRING, DEFAULT TEXT	0 of 4 (MDC not printed)
SPRING, ECS JSON	2 of 4
SPRING, ECS + TaskDecorator	3 of 4
NEST, ConsoleLogger TEXT / JSON	0 of 4
NEST, nestjs-pino	4 of 4

One request, four places that log, which lines carried req-42.

02 Request context on Nest

Take the request id from the header and let AsyncLocalStorage carry it into every log line of that request.

app.module.ts

```
LoggerModule.forRoot({
  pinoHttp: {
    genReqId: (req) => String(req.headers['x-request-id'] ?? randomUUID()),
  },
}),
```

main.ts

```
const app = await NestFactory.create(AppModule, { bufferLogs: true });
app.useLogger(app.get(Logger));
```

nestjs-pino: controller, service, after an await, in a setTimeout: 4 of 4

Run on a real machine, 2026-10-01, by scripts/verify-logging.sh in the course repository.

03 Carry the MDC across threads on Spring

A TaskDecorator copies the MDC onto Spring's task executor. Any other executor needs its own propagation.

TaskDecorator

```
@Bean
TaskDecorator mdcTaskDecorator() {
  return task -> {
    var context = MDC.getCopyOfContextMap();
    return () -> {
      if (context != null) MDC.setContextMap(context);
      try { task.run(); } finally { MDC.clear(); }
    };
  };
}
```

with the decorator: @Async carried it, CompletableFuture.runAsync did not

Run on a real machine, 2026-10-01, by scripts/verify-logging.sh in the course repository.

Before you trust your logs in production

1. Is every line JSON, with a level and a logger name?
2. Does every line of a request carry its request id?
3. Does the id survive a thread change, an await, a timer?
4. Is the id taken from the incoming header, or generated?
5. Does it reach the response, so a client can quote it?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-logging.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS logger https://docs.nestjs.com/techniques/logger

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.